

Module 19: Views

A View is a virtual table based on the result of a SQL query. Views do not store data themselves; instead, they store a query definition and display data from underlying tables whenever accessed.



Introduction

Suppose management frequently requests:

```
Employee Name  
Department Name  
Salary
```

from multiple joined tables.

Instead of writing the same JOIN query repeatedly, we can create a View.

A View behaves like a table but is actually a saved SQL query.



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand Views
 - ✓ Create Views
 - ✓ Query Views
 - ✓ Modify Views
 - ✓ Drop Views
 - ✓ Understand Security Benefits
 - ✓ Understand View Limitations
 - ✓ Use Views in Reporting Systems
-



Table of Contents

1. What is a View?
2. Why Use Views?

3. How Views Work
4. Creating Views
5. Querying Views
6. Updating Views
7. Altering Views
8. Dropping Views
9. Advantages of Views
10. Limitations of Views
11. Security with Views
12. Common Mistakes
13. Best Practices
14. Interview Questions
15. Business Analytics Examples
16. Summary
17. Practice Questions

What is a View?

A View is a virtual table created from a SQL query.

Table

Physical object.

Stores actual data.

View

Virtual object.

Stores only SQL logic.

Example

Employees

EmployeeID	EmployeeName	Salary
101	Kunal	50000
102	Rahul	60000

View:

```
CREATE VIEW EmployeeView
AS

SELECT EmployeeName,
        Salary
FROM Employees;
```

Now:

```
SELECT *
FROM EmployeeView;
```

works like a normal table.

Why Views Matter

Used in:

- Reporting Systems
 - Dashboards
 - Data Security
 - Business Intelligence
 - Data Warehouses
-

2 Why Use Views?

Imagine a report requiring:

```
Employees
JOIN Departments
JOIN Locations
```

every day.

Without View:

Write long query repeatedly.

With View:

```
SELECT *  
FROM EmployeeReport;
```

Benefits:

- ✓ Simplicity
- ✓ Reusability
- ✓ Security
- ✓ Consistency

3 How Views Work

Consider:

```
CREATE VIEW EmployeeView  
AS  
  
SELECT EmployeeName,  
       Salary  
FROM Employees;
```

SQL stores:

Query Definition

Not:

Actual Data

When:

```
SELECT *  
FROM EmployeeView;
```

runs:

SQL executes underlying query.

View Architecture

```
graph TD
    User --> View
    View --> BaseTables[Base Tables]
```



Creating Views

Syntax

```
CREATE VIEW ViewName
AS

SELECT Columns
FROM TableName;
```

Example

```
CREATE VIEW EmployeeDetails
AS

SELECT EmployeeID,
        EmployeeName,
        Salary
FROM Employees;
```

Query View

```
SELECT *  
FROM EmployeeDetails;
```

Complex View

```
CREATE VIEW EmployeeDepartmentView  
AS  
  
SELECT E.EmployeeID,  
       E.EmployeeName,  
       D.DepartmentName  
  
FROM Employees E  
  
INNER JOIN Departments D  
  
ON E.DepartmentID = D.DepartmentID;
```

Result

Joined data available as a simple table.

Querying Views

Views are queried exactly like tables.

Example

```
SELECT *  
FROM EmployeeDepartmentView;
```

Filtering

```
SELECT *  
FROM EmployeeDepartmentView  
WHERE DepartmentName = 'IT';
```

Sorting

```
SELECT *  
FROM EmployeeDepartmentView  
ORDER BY EmployeeName;
```

Key Point

Users often don't know underlying table structure.

Updating Views

Some views can be updated.

Example

```
CREATE VIEW EmployeeBasicInfo  
AS  
  
SELECT EmployeeID,  
       EmployeeName  
FROM Employees;
```

Update Through View

```
UPDATE EmployeeBasicInfo  
SET EmployeeName = 'Kunal Yadav'  
WHERE EmployeeID = 101;
```

Underlying table updates automatically.

Conditions for Updatable Views

Generally:

-  Single table

- ✓ No GROUP BY
- ✓ No Aggregate Functions
- ✓ No DISTINCT

Usually Not Updatable

```
GROUP BY
JOIN
UNION
Aggregate Functions
```

7 Altering Views

Modify existing views.

Syntax

```
ALTER VIEW ViewName
AS

SELECT ...
```

Example

Original View

```
CREATE VIEW EmployeeView
AS

SELECT EmployeeName
FROM Employees;
```

Modify

```
ALTER VIEW EmployeeView
AS
```



```
SELECT EmployeeName,  
       Salary  
FROM Employees;
```

View updated.

Dropping Views

Remove a view permanently.

Syntax

```
DROP VIEW ViewName;
```

Example

```
DROP VIEW EmployeeView;
```

Result

```
View deleted
```

Base tables remain unaffected.

Important

```
DROP VIEW EmployeeView;
```

does NOT delete:

```
Employees table
```

Data

Advantages of Views

Simplicity

Complex queries become simple.

Without View

```
SELECT ...  
FROM A  
JOIN B  
JOIN C
```

With View

```
SELECT *  
FROM SalesReport;
```

Security

Hide sensitive columns.

Example

Hide:

```
Salary  
Bank Account  
SSN
```

View exposes only required data.

Consistency

Everyone uses same logic.

Reusability

Create once.

Use many times.

Business Reporting

Very common in dashboards.

10 Limitations of Views

Performance

Complex views may execute slowly.

No Data Storage

Standard views do not store results.

Update Restrictions

Many views cannot be updated.

Dependency Issues

If underlying table changes:

```
DROP COLUMN  
RENAME COLUMN
```

View may fail.

11

Security with Views

One of the most important use cases.

Example

Employees Table

EmployeeID	EmployeeName	Salary
------------	--------------	--------

Management wants:

Employee Names Only

Create View

```
CREATE VIEW EmployeePublicView
AS

SELECT EmployeeID,
        EmployeeName
FROM Employees;
```

Users access:

EmployeePublicView

Salary remains hidden.

Real-World Uses

- HR Systems
- Finance Reports
- Customer Data Access
- Compliance Requirements

1 2 Common Mistakes

Using SELECT *

Bad:

```
CREATE VIEW EmployeeView
AS

SELECT *
FROM Employees;
```

If table changes:

View behavior changes.

Better:

```
SELECT EmployeeID,
       EmployeeName
```

Complex Nested Views

View based on View based on View.

Hard to maintain.

Forgetting Dependencies

Dropping columns can break views.

Performance Problems

Large joins inside views.

1 3 Best Practices

Use Explicit Column Names

Good:

```
SELECT EmployeeID,  
       EmployeeName
```

Use Meaningful Names

Good:

```
EmployeeReportView  
SalesSummaryView
```

Document Purpose

Every view should have clear business purpose.

Avoid Deep Nesting

Keep view hierarchy simple.

Use Views for Security

Hide sensitive data.

Professional Example

```
CREATE VIEW EmployeeReportView  
AS  
  
SELECT E.EmployeeID,  
       E.EmployeeName,  
       D.DepartmentName  
  
FROM Employees E
```

```
INNER JOIN Departments D  
ON E.DepartmentID = D.DepartmentID;
```

1 4 Common Interview Questions

What is a View?

A virtual table based on a query.

Does a View Store Data?

Normally no.

Stores query definition.

Difference Between Table and View?

Table stores data.

View stores query logic.

Can Views Be Updated?

Some can.

Depends on complexity.

Why Use Views?

Security, simplicity, reusability.

What Happens When a View Is Dropped?

Only view removed.

Underlying tables remain.

1 5 Business Analytics Examples

Employee Report

```
CREATE VIEW EmployeeReport
AS

SELECT EmployeeID,
        EmployeeName,
        Salary

FROM Employees;
```

Department Dashboard

```
CREATE VIEW DepartmentDashboard
AS

SELECT DepartmentID,
        COUNT(*) AS Employees

FROM Employees

GROUP BY DepartmentID;
```

Sales Summary

```
CREATE VIEW SalesSummary
AS

SELECT ProductID,
        SUM(SalesAmount) AS TotalSales

FROM Sales

GROUP BY ProductID;
```

Customer Analytics

```
CREATE VIEW CustomerAnalytics
AS

SELECT CustomerID,
        COUNT(*) AS Orders

FROM Orders

GROUP BY CustomerID;
```

KPI Reporting Layer

Views are often created specifically for:

```
Power BI
Tableau
Looker
SSRS
Excel Dashboards
```



Summary

In this module, you learned:

- ✓ CREATE VIEW
- ✓ Querying Views
- ✓ Updating Views
- ✓ ALTER VIEW
- ✓ DROP VIEW
- ✓ Security Benefits
- ✓ View Limitations
- ✓ Reporting Use Cases



Practice Questions

Theory

1. What is a View?
 2. Why are Views used?
 3. Does a View store data?
 4. Difference between Table and View?
 5. How do you create a View?
 6. How do you modify a View?
 7. How do you delete a View?
 8. What are View advantages?
 9. What are View limitations?
 10. How do Views improve security?
-

Practical Exercises

Task 1

Create:

```
EmployeeView
```

showing employee details.

Task 2

Create a JOIN-based View.

Task 3

Modify a View using:

```
ALTER VIEW
```

Task 4

Drop a View.

Task 5

Create a security-focused View hiding salary data.

Task 6

Create a summary View using:

GROUP BY

Challenge Project

Create:

Employees
Departments
Orders
Customers
Sales

Build:

- Employee Report View
- Department Dashboard View
- Sales Summary View
- Customer Analytics View

for a reporting environment.



Next Module

→ Module 20: Indexes

Topics Covered:

- What are Indexes?
- Clustered Index
- Non-Clustered Index
- Unique Index
- Composite Index
- Index Performance
- Index Maintenance
- Query Optimization
- Execution Plans